MANUAL DE USUARIO

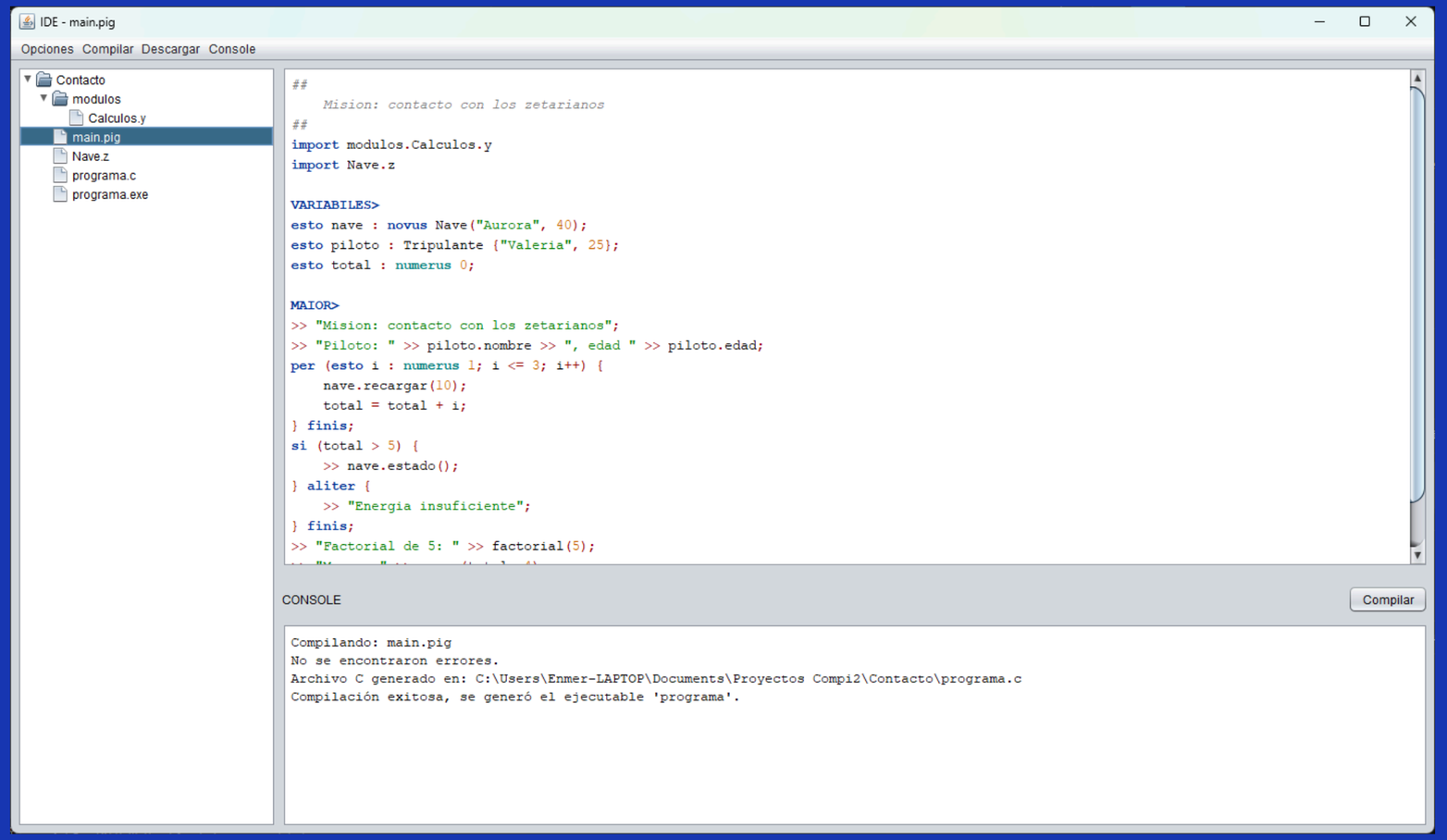
Proyecto 1: Contacto 3xtrat3rr3str3D

Enmer Oswaldo Sandoval Mazariegos 3301795511201

QUÉ HACE EL SISTEMA?

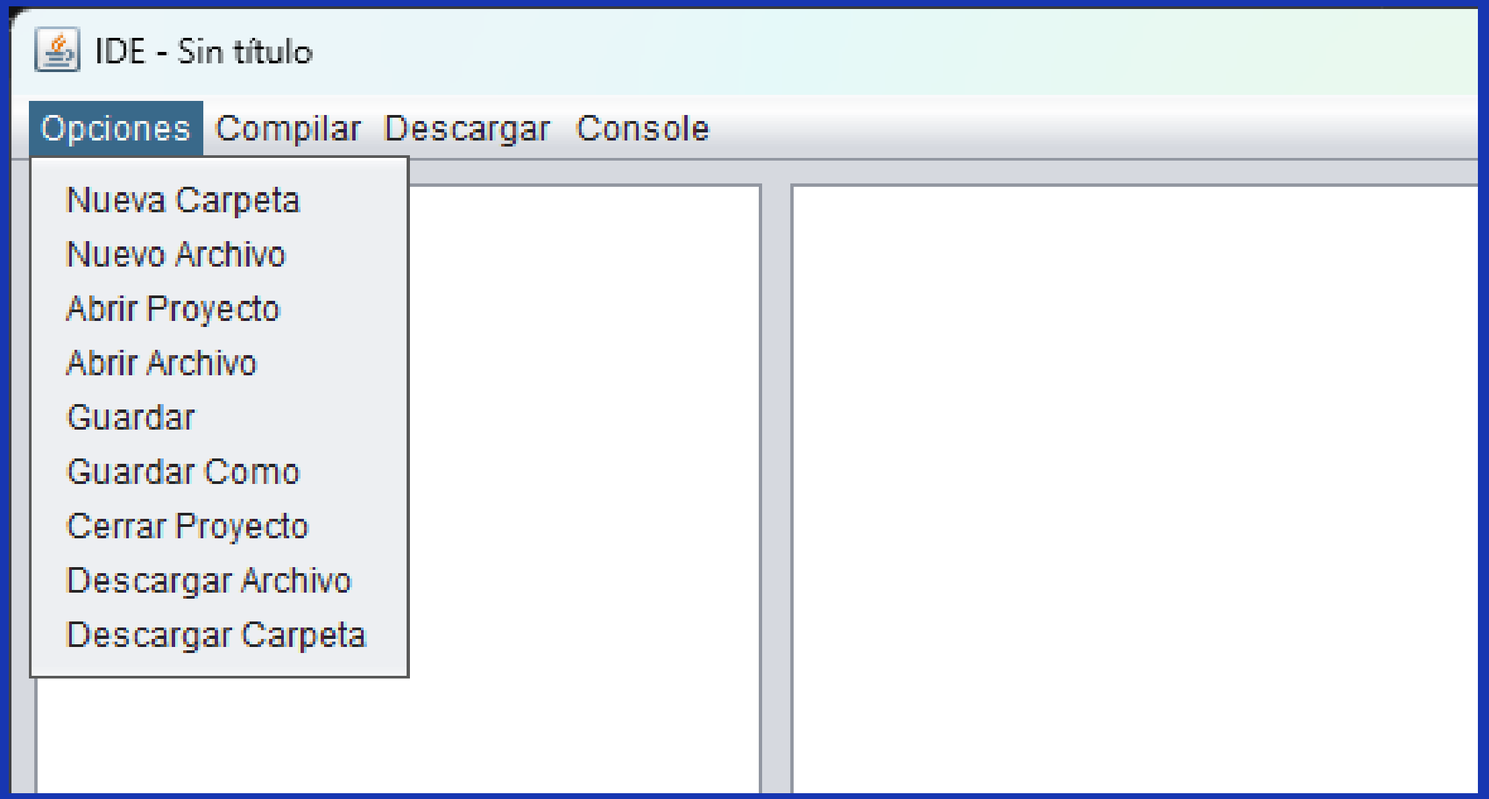
El sistema es un compilador para tres lenguajes: Y?, Zetariano y Pig Latin. Revisa el código de forma léxica, sintáctica y semántica, y genera cuartetas, código de 3 Direcciones y código C que se compila con gcc

INTERFAZ



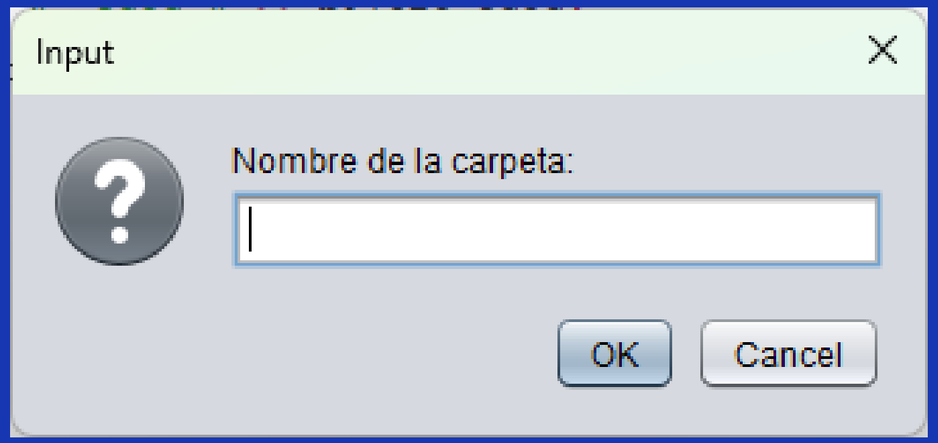
Manejamos un árbol de archivos, un editor con colores y una consola donde se muestran los mensajes

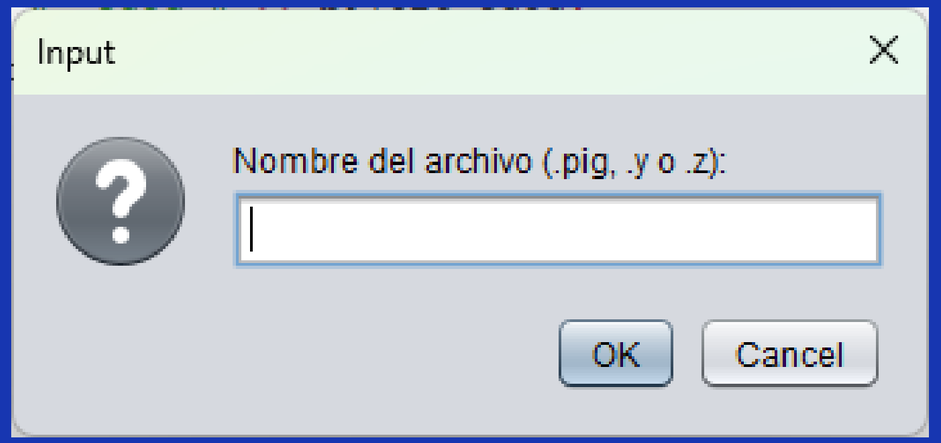
MENÚ OPCIONES



- Nueva Carpeta: crea un proyecto o una subcarpeta
- Nuevo Archivo: crea un archivo .pig, .y o .z
- Abrir Proyecto / Abrir Archivo
- Guardar / Guardar Como
- Cerrar Proyecto
- Descargar Archivo / Descargar Carpeta

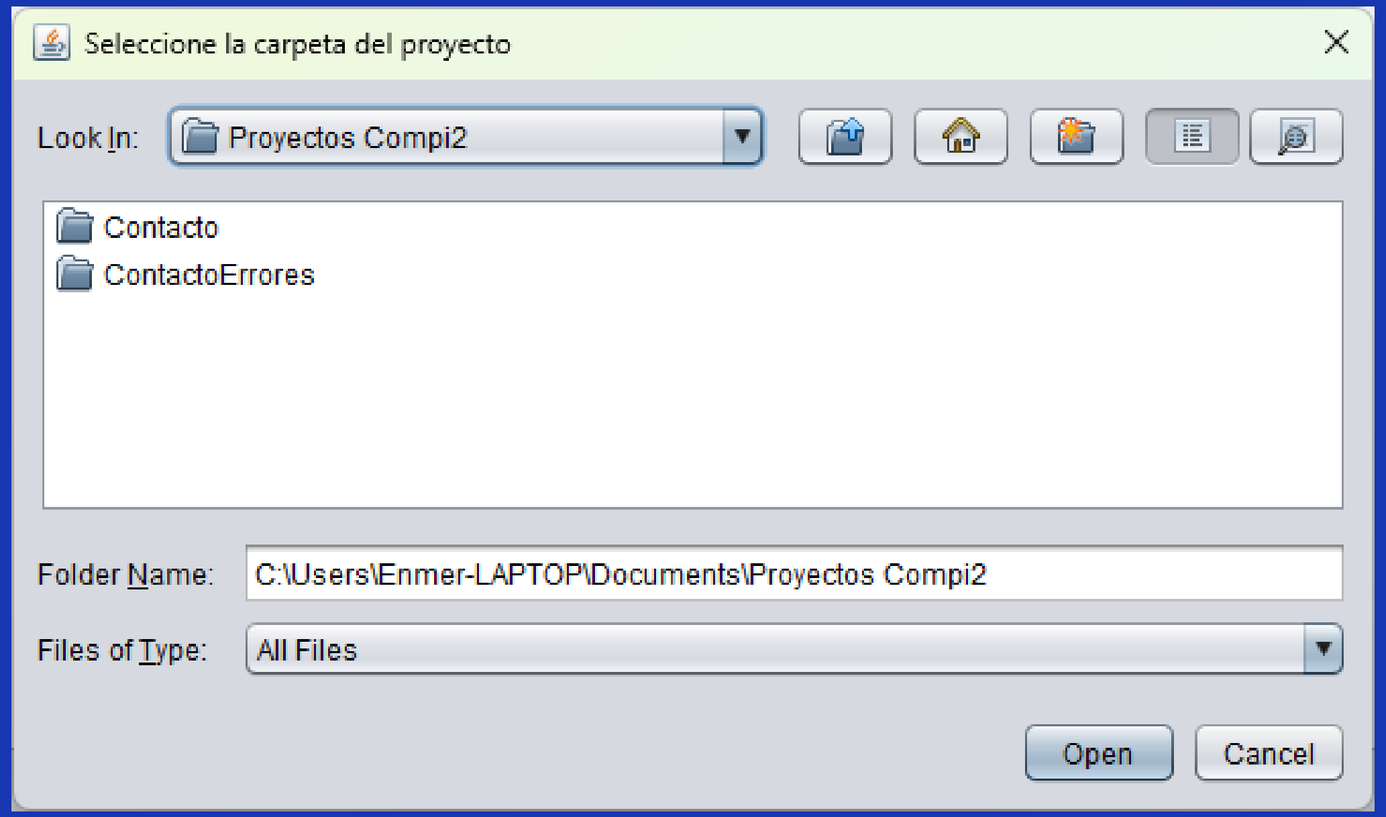
NUEVA CARPETA Y NUEVO ARCHIVO





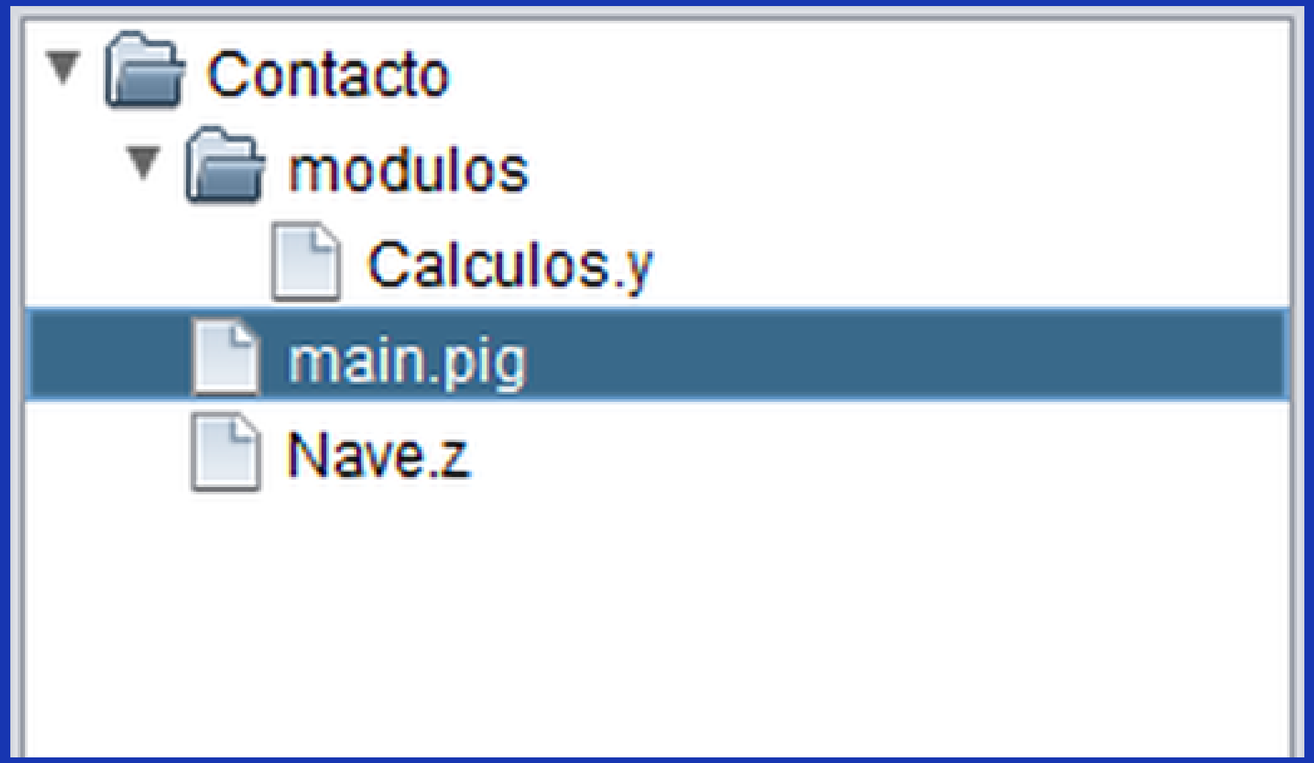
La carpeta se crea dentro de la carpeta seleccionada en el árbol. El archivo debe terminar en .pig, .y o .z

JFileChooser para abrir un proyecto



Se selecciona la carpeta del proyecto y se carga en el árbol

NUESTRO ÁRBOL DE ARCHIVOS



Con un clic sobre un archivo se abre en el editor

CÓDIGO CON COLORES

```
##
    Mision: contacto con los zetarianos
##
import modulos.Calculos.y
import Nave.z

VARIABILES>
esto nave : novus Nave("Aurora", 40);
esto piloto : Tripulante {"Valeria", 25};
esto total : numerus 0;

MAIOR>
>> "Mision: contacto con los zetarianos";
>> "Piloto: " >> piloto.nombre >> ", edad " >> piloto.edad;
per (esto i : numerus 1; i <= 3; i++) {
    nave.recargar(10);
    total = total + i;
} finis;
si (total > 5) {
    >> nave.estado();
} aliter {
    >> "Energia insuficiente";
} finis;
>> "Factorial de 5: " >> factorial(5);
```

El código se colorea en tiempo real mientras se escribe

Y? Y ZETARIANO

```
%estructuras
estructura Tripulante:
    cadena nombre
    entero edad

%funciones
// calcula n! de forma recursiva
definir factorial(entero n) -> entero:
    si(n <= 1) entonces
        retornar 1
    retornar n * factorial(n - 1)

definir mayor(entero a, entero b) -> entero:
    si(a > b) entonces
        retornar a
    contrario
        retornar b
```

```
public class Nave {
    String nombre;
    int energia;

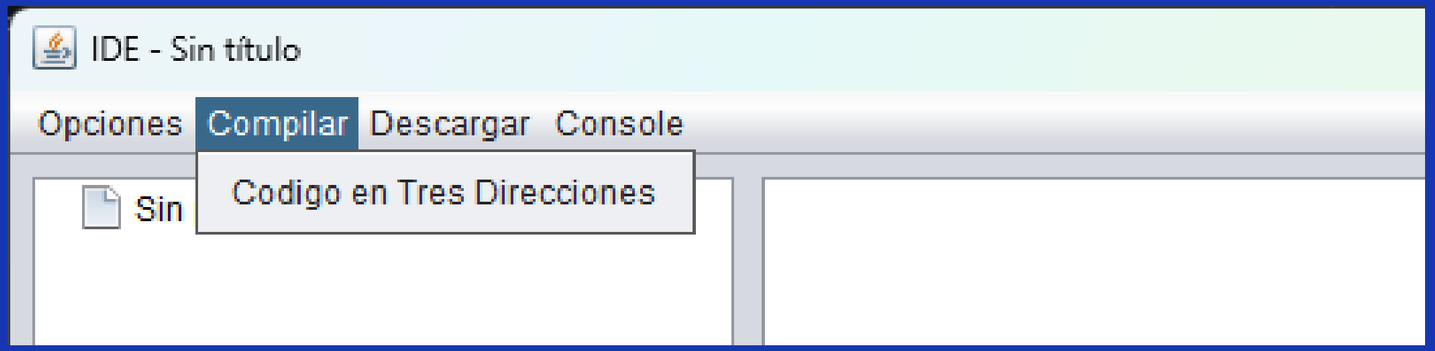
    public Nave(String nombre, int energia) {
        this.nombre = nombre;
        this.energia = energia;
    }

    // suma energia a la nave
    public void recargar(int cantidad) {
        energia = energia + cantidad;
    }

    public String estado() {
        if (energia > 50) {
            return nombre + " lista para el despegue con " + energia;
        }
        return nombre + " necesita energia";
    }
}
```

Cada lenguaje tiene sus propios colores

COMPILAR

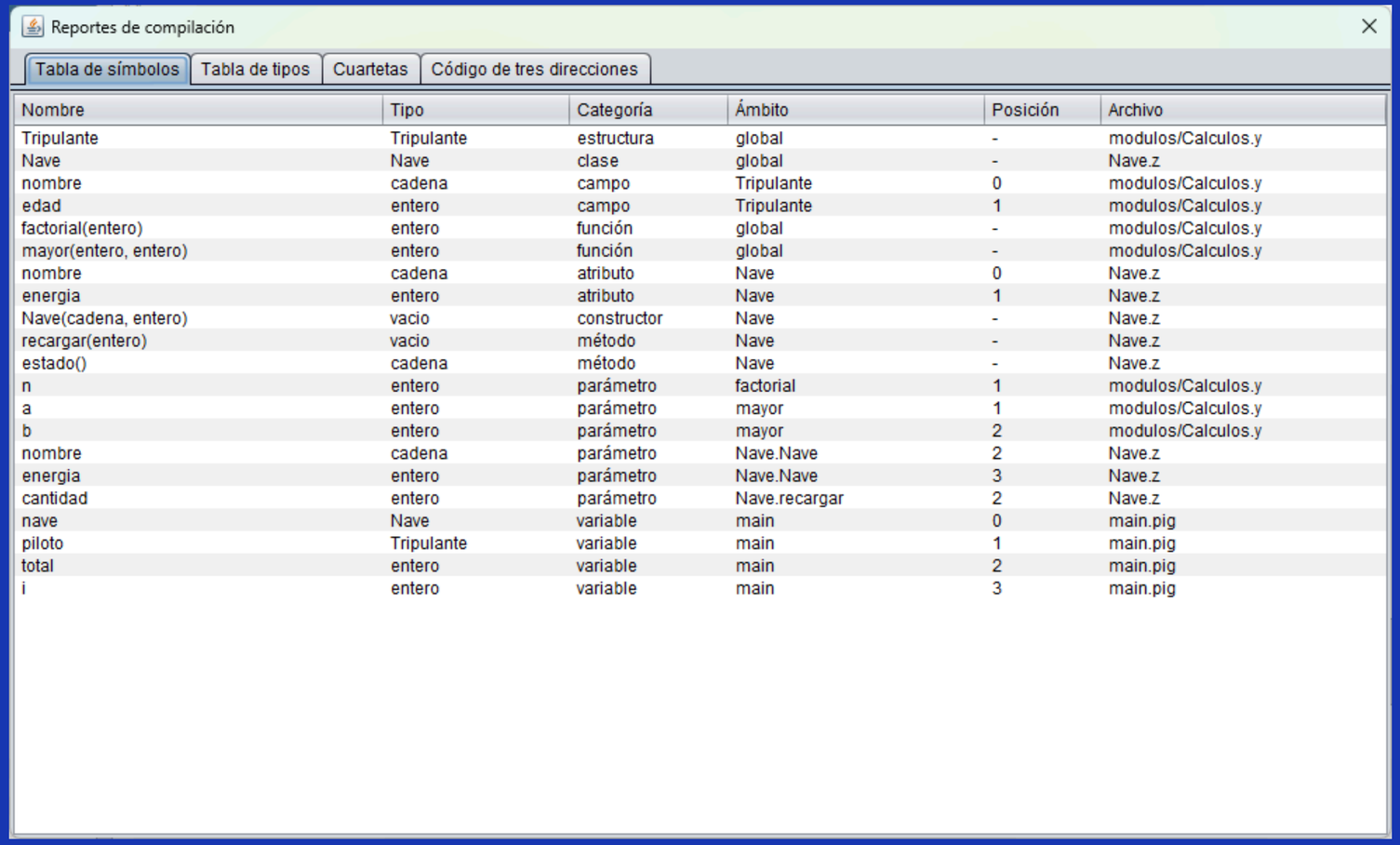



```
Compilando: main.pig
No se encontraron errores.
Archivo C generado en: C:\Users\Enmer-LAPTOP\Documents\Proyectos Compi2\Contacto\programa.c
Compilación exitosa, se generó el ejecutable 'programa'.
```

Se compila con el botón Compilar o con Compilar → Codigo en Tres Direcciones.

Console → Clean Console limpia la consola

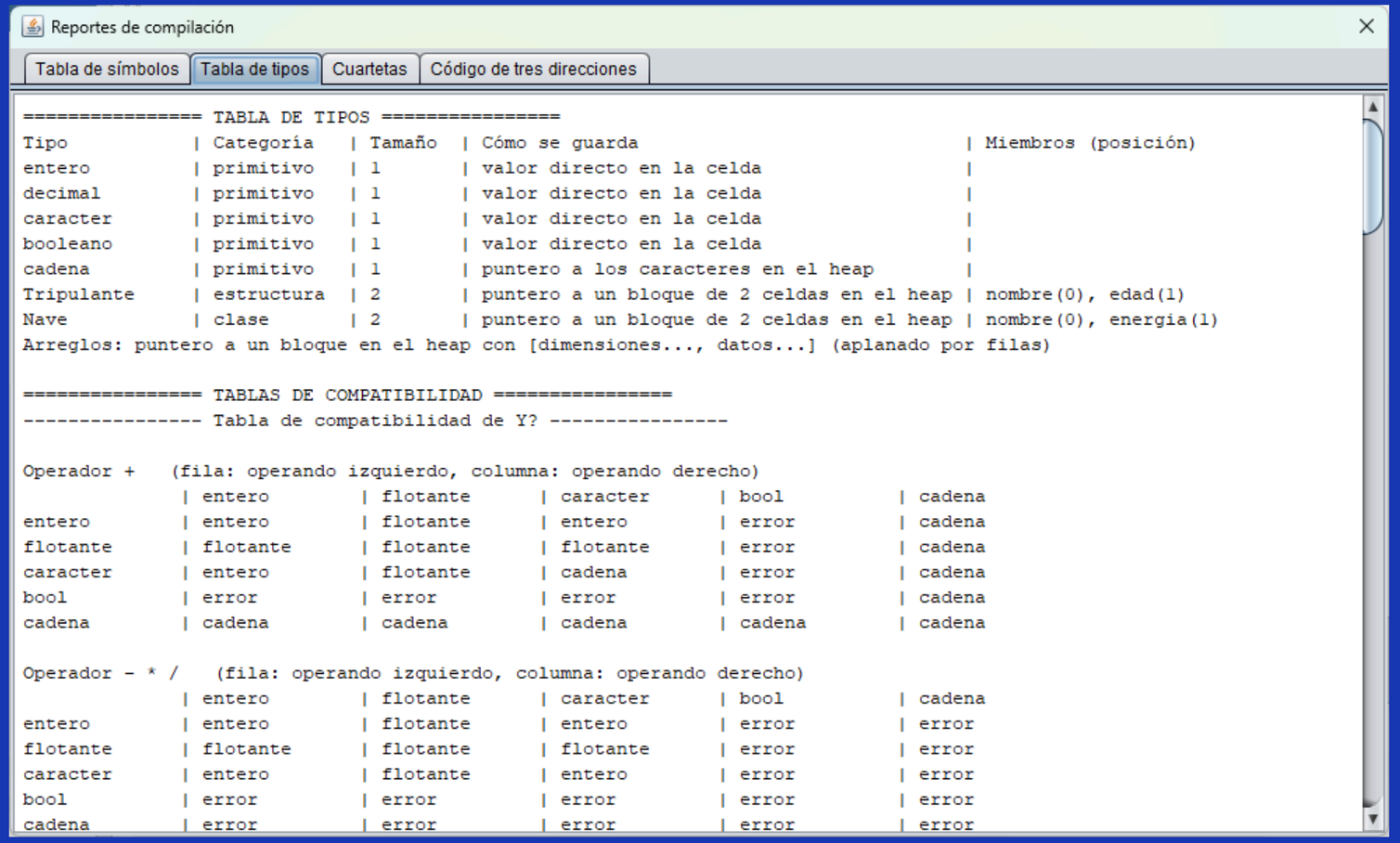
Tabla de Símbolos



Nombre	Tipo	Categoría	Ámbito	Posición	Archivo
Tripulante	Tripulante	estructura	global	-	modulos/Calculos.y
Nave	Nave	clase	global	-	Nave.z
nombre	cadena	campo	Tripulante	0	modulos/Calculos.y
edad	entero	campo	Tripulante	1	modulos/Calculos.y
factorial(entero)	entero	función	global	-	modulos/Calculos.y
mayor(entero, entero)	entero	función	global	-	modulos/Calculos.y
nombre	cadena	atributo	Nave	0	Nave.z
energia	entero	atributo	Nave	1	Nave.z
Nave(cadena, entero)	vacio	constructor	Nave	-	Nave.z
recargar(entero)	vacio	método	Nave	-	Nave.z
estado()	cadena	método	Nave	-	Nave.z
n	entero	parámetro	factorial	1	modulos/Calculos.y
a	entero	parámetro	mayor	1	modulos/Calculos.y
b	entero	parámetro	mayor	2	modulos/Calculos.y
nombre	cadena	parámetro	Nave.Nave	2	Nave.z
energia	entero	parámetro	Nave.Nave	3	Nave.z
cantidad	entero	parámetro	Nave.recargar	2	Nave.z
nave	Nave	variable	main	0	main.pig
piloto	Tripulante	variable	main	1	main.pig
total	entero	variable	main	2	main.pig
i	entero	variable	main	3	main.pig

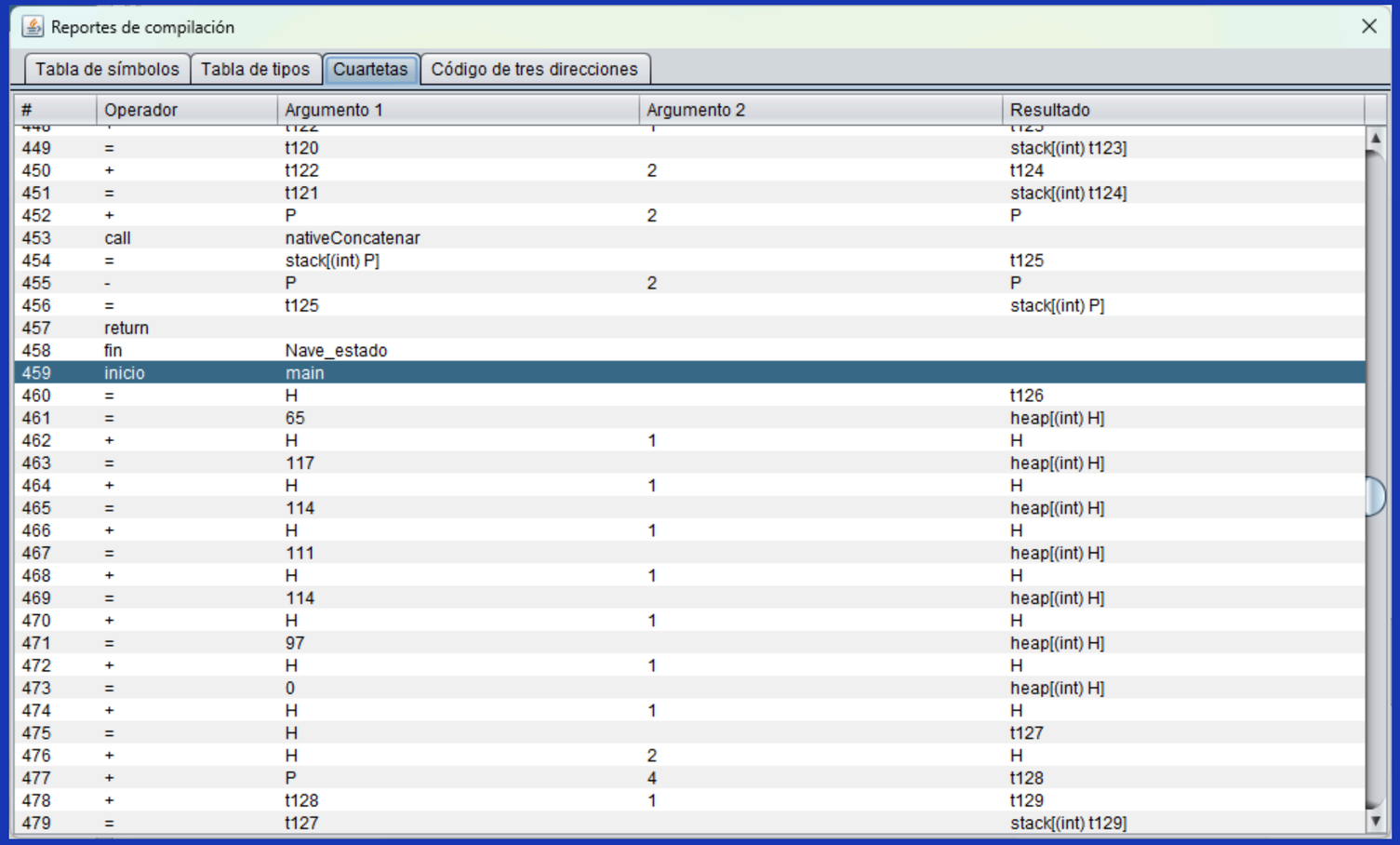
Al compilar sin errores se abren los reportes

Tabla de Tipos



Cómo se guarda cada tipo y las tablas de compatibilidad

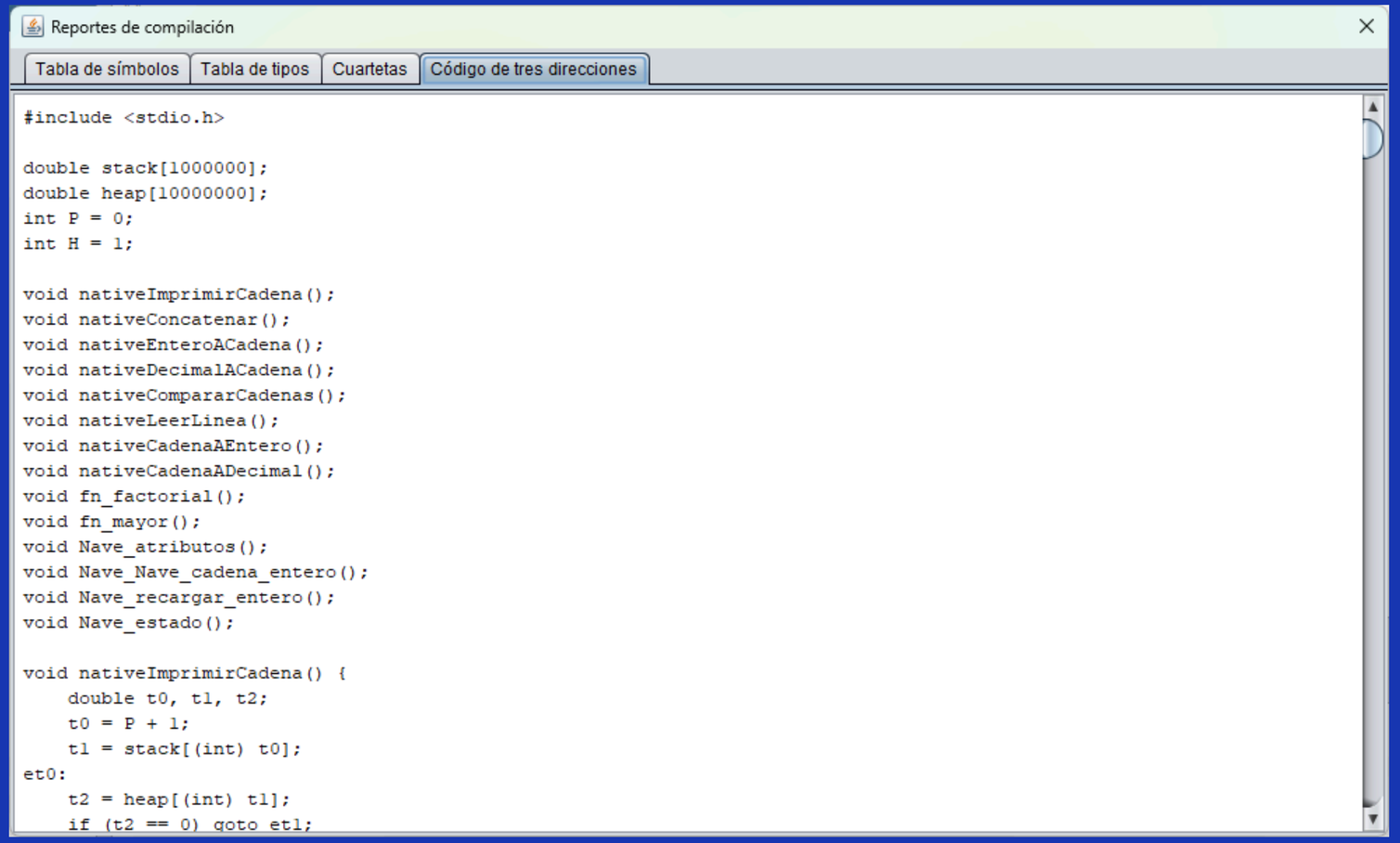
Cuartetas



#	Operador	Argumento 1	Argumento 2	Resultado
449	=	t120		stack[(int) t123]
450	+	t122	2	t124
451	=	t121		stack[(int) t124]
452	+	P	2	P
453	call	nativeConcatenar		
454	=	stack[(int) P]		t125
455	-	P	2	P
456	=	t125		stack[(int) P]
457	return			
458	fin	Nave_estado		
459	inicio	main		
460	=	H		t126
461	=	65		heap[(int) H]
462	+	H	1	H
463	=	117		heap[(int) H]
464	+	H	1	H
465	=	114		heap[(int) H]
466	+	H	1	H
467	=	111		heap[(int) H]
468	+	H	1	H
469	=	114		heap[(int) H]
470	+	H	1	H
471	=	97		heap[(int) H]
472	+	H	1	H
473	=	0		heap[(int) H]
474	+	H	1	H
475	=	H		t127
476	+	H	2	H
477	+	P	4	t128
478	+	t128	1	t129
479	=	t127		stack[(int) t129]

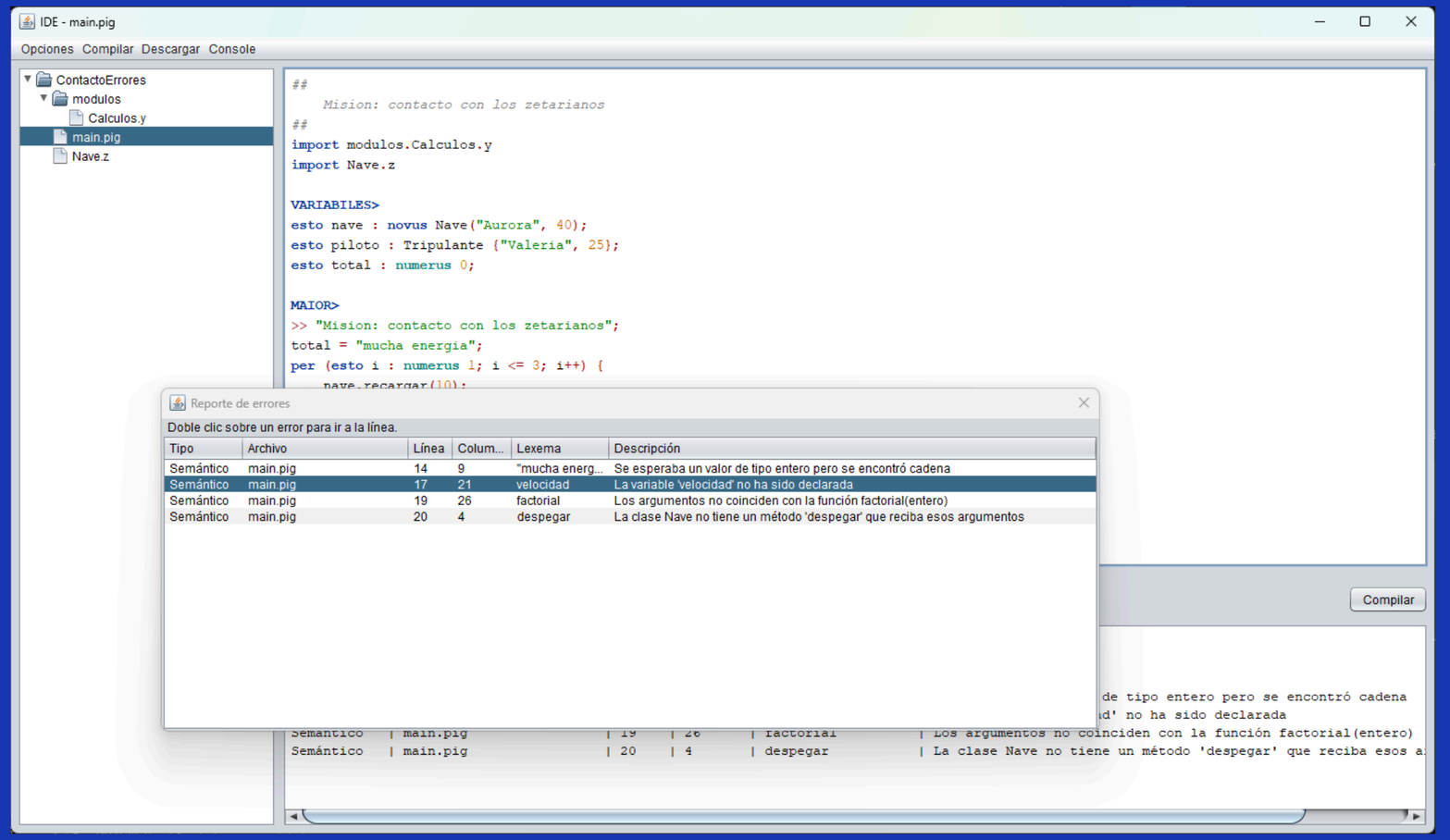
Operador, argumento 1, argumento 2 y resultado

Código de 3 Direcciones



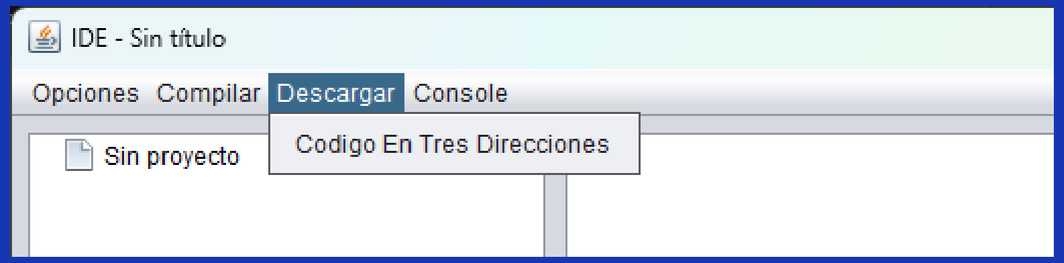
El código C que se guarda como programa.c

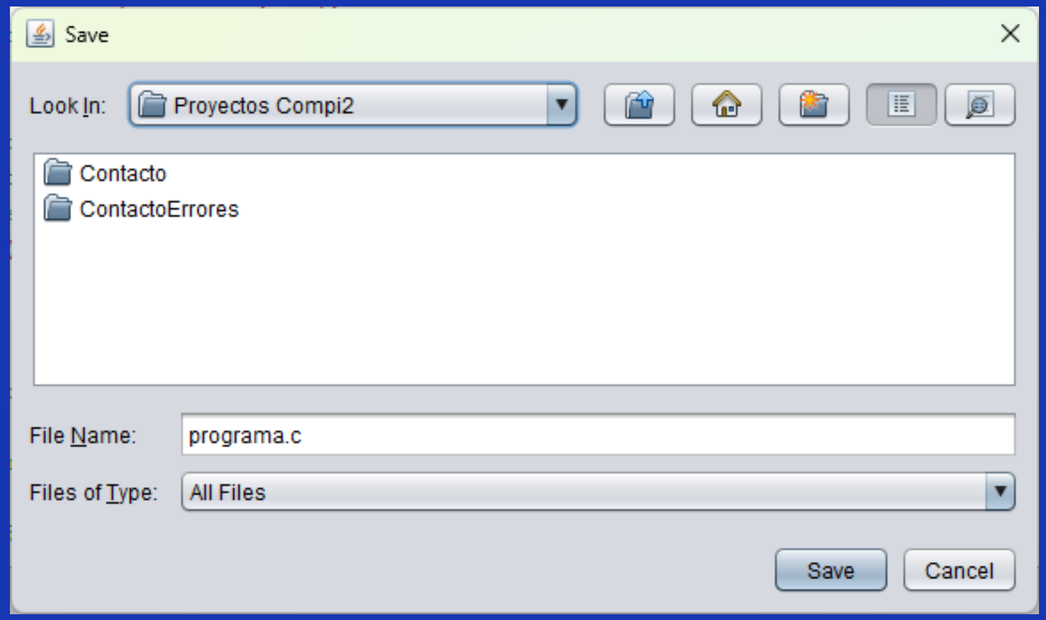
Tabla de Errores



Tipo, archivo, línea, columna, lexema y descripción. Con doble clic se va a la línea del error

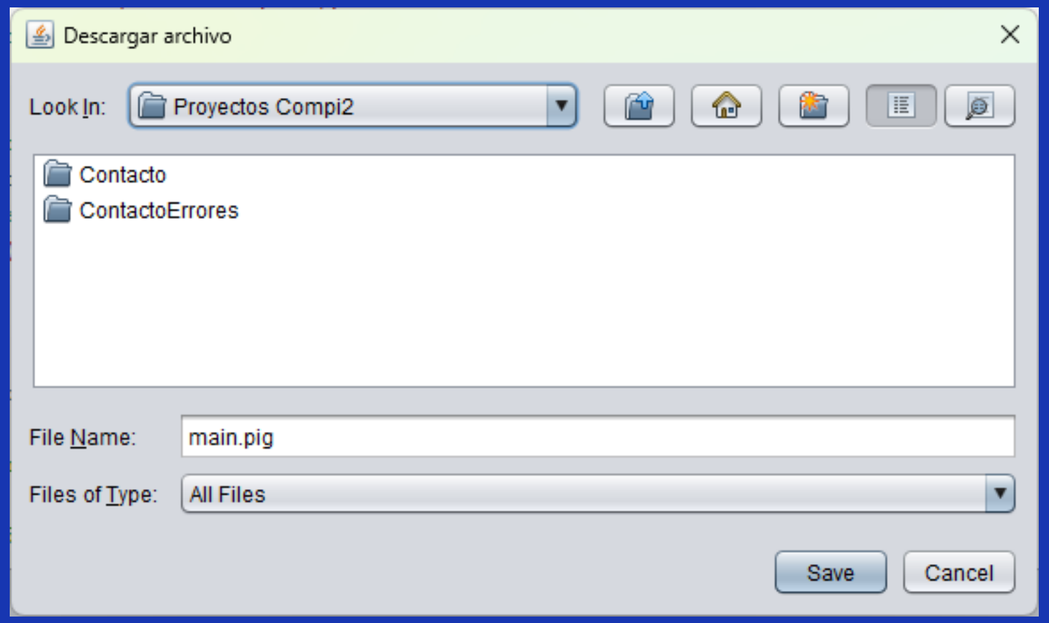
DESCARGAR CÓDIGO

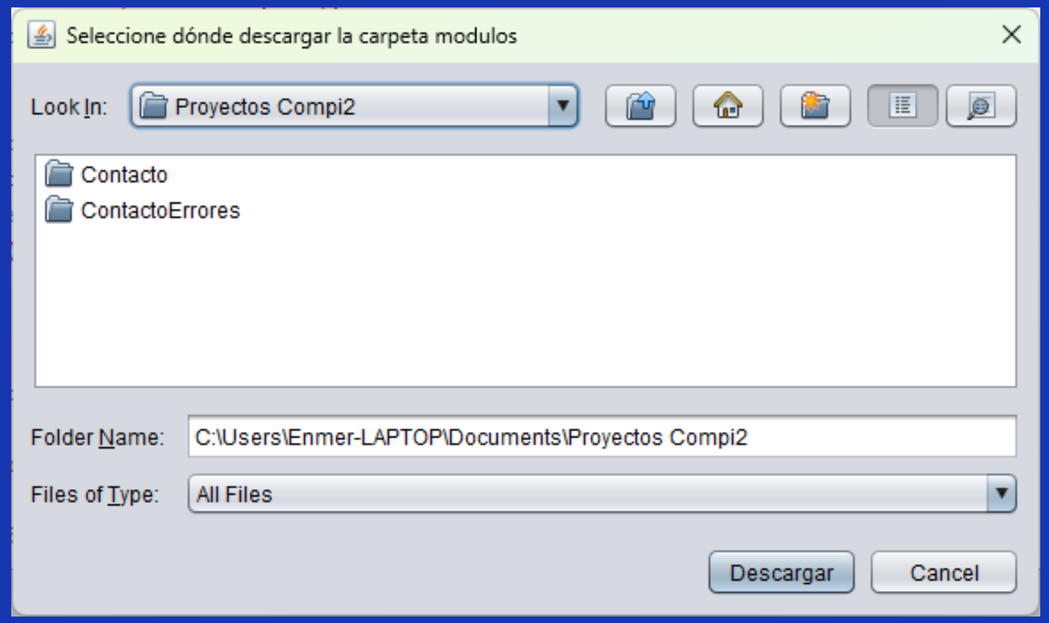




Descargar → Codigo En Tres Direcciones guarda el programa.c donde usted elija

DESCARGAR ARCHIVO Y CARPETA





Se descarga el archivo o la carpeta seleccionada en el árbol

GRACIAS